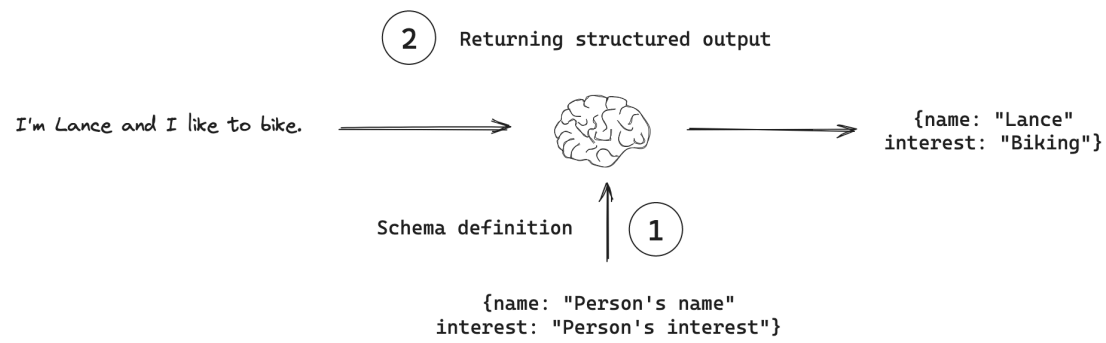
Open on GitHub

结构化输出

对于许多应用程序（例如聊天机器人），模型需要直接用自然语言响应用户。然而，在某些情况下，我们需要模型以*结构化格式*输出。例如，我们可能希望将模型输出存储在数据库中，并确保输出符合数据库模式。这种需求催生了结构化输出的概念，即可以指示模型使用特定的输出结构进行响应。



关键

1. **模式定义：**输出结构表示为模式，可以通过多种方式定义。
2. **返回结构化输出：**模型被赋予此模式，并被指示返回符合该模式的输出。

建议

此伪代码演示了使用结构化输出时的推荐工作流程。LangChain 提供了一种方法，`with_structured_output()`可以自动执行将架构绑定到模型并解析输出的过程。此辅助函数适用于所有支持结构化输出的模型提供程序。

```
# Define schema
schema = {"foo": "bar"}
# Bind schema to model
model_with_structure =
model.with_structured_output(schema)
# Invoke the model to produce structured output that
matches the schema
structured_output =
model_with_structure.invoke(user_input)
```

工具顺序很重要

当将结构化输出与其他工具结合使用时，**请先**绑定工具，然后应用结构化输出：

```
# Correct
model_with_tools = model.bind_tools([tool1, tool2])
structured_model =
model_with_tools.with_structured_output(schema)

# Incorrect - will cause tool resolution errors
structured_model =
```

```
model.with_structured_output(schema)
broken_model = structured_model.bind_tools([tool1,
tool2])
```

架构

核心概念是模型响应的输出结构需要以某种方式表示。虽然您可以使用的对象类型取决于您正在使用的模型，但 Python 中通常允许或推荐使用一些常见的对象类型来进行结构化输出。

结构化输出最简单、最常见的格式是类似 JSON 的结构，在 Python 中可以表示为字典 (dict) 或列表 (list)。当工具需要原始、灵活且开销极小的结构化数据时，通常会直接使用 JSON 对象（在 Python 中为 dict）。

```
{
  "answer": "The answer to the user's question",
  "followup_question": "A followup question the user
could ask"
}
```

第二个例子是，Pydantic对于定义结构化输出模式特别有用，因为它提供了类型提示和验证。以下是 Pydantic 模式的示例：

```
from pydantic import BaseModel, Field
class ResponseFormatter(BaseModel):
    """Always use this tool to structure your response
to the user."""
    answer: str = Field(description="The answer to the
user's question")
    followup_question: str = Field(description="A
followup question the user could ask")
```

返回结构化

定义好模式后，我们需要一种方法来指示模型使用它。虽然一种方法是将此模式包含在提示中，并*礼貌地请求*模型使用它，但不建议这样做。还有一些更强大的方法可以利用模型提供程序 API 中的原生功能。

使用工具

许多模型提供商都支持工具调用，我们在工具调用指南中更详细地讨论了这个概念。简而言之，工具调用涉及将工具绑定到模型，并且在适当的时候，模型可以*决定*调用此工具并确保其响应符合工具的模式。考虑到这一点，核心概念很简单：*只需将我们的模式作为工具绑定到模型即可！*

`ResponseFormatter`以下是使用上面定义的模式的 示例：

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4o", temperature=0)
# Bind responseformatter schema as a tool to the model
model_with_tools =
model.bind_tools([ResponseFormatter])
```

```
# Invoke the model
ai_msg = model_with_tools.invoke("What is the
powerhouse of the cell?")
```

工具调用的参数已被提取为字典。此字典可以选择解析为 Pydantic 对象，以匹配我们原始的`ResponseFormatter`模式。

```
# Get the tool call arguments
ai_msg.tool_calls[0]["args"]
{'answer': "The powerhouse of the cell is the
mitochondrion. Mitochondria are organelles that
generate most of the cell's supply of adenosine
triphosphate (ATP), which is used as a source of
chemical energy.",
 'followup_question': 'What is the function of ATP in
the cell?'}
# Parse the dictionary into a pydantic object
pydantic_object =
ResponseFormatter.model_validate(ai_msg.tool_calls[0]
["args"])
```

JSON

除了工具调用之外，一些模型提供程序还支持名为 的功能。该功能支持 JSON 模式定义作为输入，并强制模型生成符合规范的 JSON 输出。您可以在此处`JSON mode`找到支持 JSON 模式的模型提供程序表格。以下是如何在 OpenAI 中使用 JSON 模式的示例：

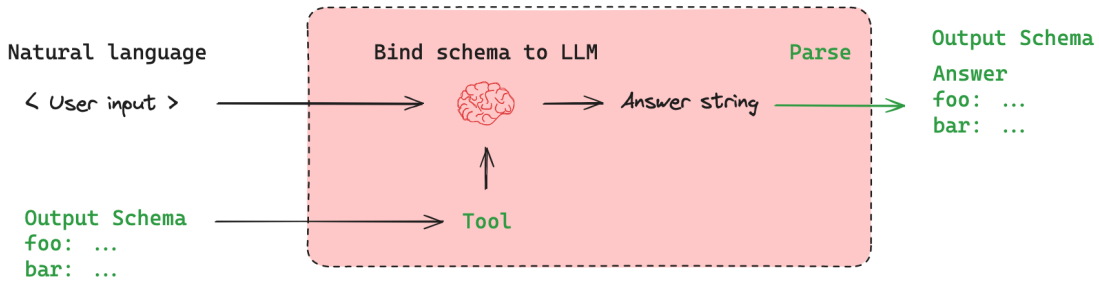
```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-
4o").with_structured_output(method="json_mode")
ai_msg = model.invoke("Return a JSON object with key
'random_ints' and a value of 10 random ints in [0-99]")
ai_msg
{'random_ints': [45, 67, 12, 34, 89, 23, 78, 56, 90,
11]}
```

结构化输出

使用上述方法生成结构化输出时存在一些挑战：

1. 当使用工具调用时，需要将工具调用参数从字典解析回原始模式。
2. 此外，当我们想要强制执行结构化输出时，需要指示模型*始终使用该工具，这是一个特定于提供商的设置。*
3. 当使用JSON模式时，需要将输出解析为JSON对象。

考虑到这些挑战，LangChain 提供了一个辅助函数（`with_structured_output()`）来简化流程。



这既将模式绑定到模型作为工具，又将输出解析为指定的输出模式。

```
# Bind the schema to the model
model_with_structure =
model.with_structured_output(ResponseFormatter)
# Invoke the model
structured_output = model_with_structure.invoke("What
is the powerhouse of the cell?")
# Get back the pydantic object
structured_output
ResponseFormatter(answer="The powerhouse of the cell is
the mitochondrion. Mitochondria are organelles that
generate most of the cell's supply of adenosine
triphosphate (ATP), which is used as a source of
chemical energy.", followup_question='What is the
function of ATP in the cell?')
```

进一步阅读

有关使用方法的更多详细信息，请参阅我们的操作指南。

编辑此页面